

imageGen.py x

```
1 import sys
2 import os
3 from PIL import Image, ImageOps, ImageChops
4 from io import BytesIO
5 import requests
6
7
8 # set our images to a variablke
9 image1 = Image.open('image1.jpg')
10 image2 = Image.open('image2.jpg')
11 # Blend the images using the Multiply mode
12 image3 = Image.open('moon_image.jpg')
13 image4 = Image.open('night_sky.jpg')
14
15 def custom_blend(image1, image2):
16     return ImageChops.add_modulo(image1, image2).point(lambda x: x // 2)
17
18 # Blend the images using the custom blend mode
19 result = custom_blend(image1, image2)
20 result2= custom_blend(image3, image4)
21 # Save the blended image to a file
22 result.save('custom_blend.jpg')
23 result2.save('custom_blend2.jpg')
```

100% (23:34)

Terminal x

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 5.19.0-1028-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch
 *
 * Welcome to the Codio Terminal!
 *
```

Advanced Blending Techniques

Click here to refresh your custom blend of image 2

This is the updated visual of the moon and night sky when using a custom blend.

## Custom Blend Function

The `custom_blend` function returns the resulting image, which is the blended image with the average color value of the two input images. The `custom_blend` function takes two images, `image1` and `image2`, as inputs. Its purpose is to blend these two images together by calculating the average color value of each corresponding pixel from both images.

`ImageChops.add_modulo(image1, image2)` adds the pixel values of `image1` and `image2` together. It does this by adding the values of corresponding pixels in each image while ensuring that the resulting values don't exceed the maximum allowed value for each color channel. In this case, the maximum allowed value is 255 for an 8-bit image. The `add_modulo` function ensures that if the sum of two pixel values exceed 255, the result wraps around from 0, in a similar fashion to how numbers wrap around when using the modulo operation.

The resulting image from the `add_modulo()` operation has pixel values that are double the average of the original images' pixel values. In order to get the true average, we need to divide these values by 2. We do this using the `point()` function, which applies a given function to each pixel value in the image. In this case, we use a lambda function `lambda x: x // 2`, which takes an input `x` (the pixel value) and returns the integer division of x by 2 (effectively dividing the pixel value by 2).

What does the `ImageChops.add_modulo(image1, image2)` function do?

☐ Multiplies the pixel values of image1 and image2.

☒ Adds the pixel values of image1 and image2 together, ensuring that the resulting values don't exceed the maximum allowed value for each color channel.

☐ Averages the pixel values of image1 and image2.

☐ Blends image1 and image2 using a custom gradient mask.

The `ImageChops.add_modulo(image1, image2)` function adds the pixel values of image1 and image2 together while ensuring that the resulting values don't exceed the maximum allowed value for each color channel. In the case of an 8-bit image, the maximum allowed value is 255. If the sum of two pixel values would exceed 255, the result wraps around from 0.

Check It!

Next